

# Deep Research: Avatrada 57 Topic 047

---

## Engineering Report: Tiered Storage Architecture

---

### Executive Summary

This report provides a deep-dive analysis of the specified three-tiered storage architecture for a trading system. The architecture is designed to balance performance, cost, and data accessibility by segregating data based on its access frequency and latency requirements. The "Hot" tier uses Redis for instantaneous access to live operational data. The "Warm" tier uses PostgreSQL for transactional data and recent history, offering a balance of performance and rich query capabilities. The "Cold" tier uses Google Cloud Storage (GCS) for long-term, cost-effective archival of historical data. This report deconstructs the architecture, provides a concrete implementation strategy including a data migration script, and offers a critical analysis of potential failure modes and optimizations.

---

### 1. Technical Deconstruction

The proposed system employs a classic data lifecycle management pattern, moving data from high-cost, high-performance storage to low-cost, lower-performance storage as its access value diminishes over time.

#### 1.1. Hot Tier: Redis

- **Technology:** Redis (In-Memory Data Store)
- **Data:** Order State, Live Signals
- **Mechanism:** Redis stores data in RAM, enabling microsecond-level latency for read and write operations. This is critical for a trading system where

the status of an order or the value of a market signal must be accessed and updated nearly instantaneously by multiple services.

- **Rationale:**

- **Performance:** The primary driver is speed. In-memory storage eliminates disk I/O bottlenecks.
- **Data Structures:** Redis provides versatile data structures ideal for this use case. A `HASH` is perfect for storing the fields of an `Order State` (e.g., `order_id`, `status`, `price`, `quantity`). `STREAMS` or `PUBSUB` are highly effective for broadcasting `Live Signals` to various consumers.
- **Volatility:** This data is ephemeral. An order's state is only "hot" while it's active. Once filled or canceled, its immediate relevance drops, making it a candidate for migration to a persistent store.

## 1.2. Warm Tier: PostgreSQL

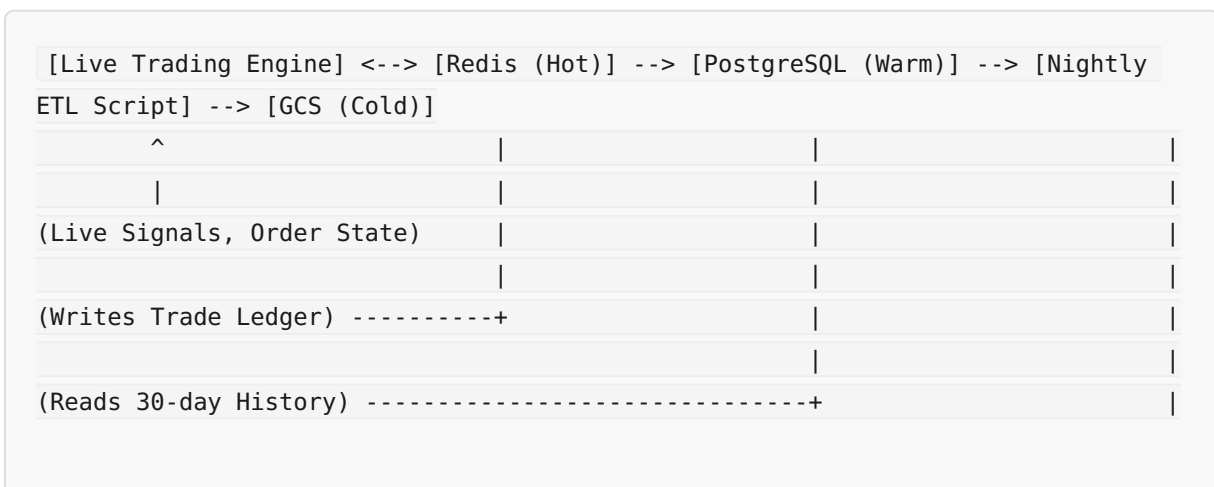
- **Technology:** PostgreSQL (Relational Database)
- **Data:** `Trade Ledger`, `30-day History`
- **Mechanism:** PostgreSQL is an ACID-compliant relational database that stores data on persistent disk (typically SSDs). It provides strong transactional guarantees, ensuring the integrity of financial records like the `Trade Ledger`. Its SQL interface allows for complex queries, aggregations, and reporting on recent historical data.
- **Rationale:**
  - **Integrity:** For a `Trade Ledger`, transactional guarantees (Atomicity, Consistency, Isolation, Durability) are non-negotiable. PostgreSQL ensures that trades are recorded accurately and reliably.
  - **Queryability:** The 30-day history of signals and trades needs to be available for analysis, dashboarding, and tactical backtesting. SQL is the industry standard for this type of relational querying.
  - **Cost/Performance Balance:** While slower than Redis, a well-tuned Postgres instance on modern SSDs provides millisecond-level query times, which is sufficient for non-real-time operations. It is significantly cheaper than storing gigabytes or terabytes of data in RAM.

### 1.3. Cold Tier: Google Cloud Storage (GCS)

- **Technology:** Google Cloud Storage (Object Storage)
- **Data:** CSV Archives (specified), optimized to Parquet format.
- **Mechanism:** GCS is a durable, scalable, and low-cost object storage service. Data is stored as immutable objects (files) in "buckets". While the source specifies CSV, we will implement with Parquet, a columnar storage format that is highly compressed and optimized for analytical queries.
- **Rationale:**
  - **Cost:** GCS is exceptionally cheap for long-term storage, orders of magnitude less expensive than a managed database for the same volume of data.
  - **Durability & Scalability:** GCS offers extremely high durability guarantees (99.999999999%) and virtually infinite scalability. It's ideal for archiving vast amounts of historical tick data without worrying about disk space.
  - **Analytical Integration:** Storing data as Parquet files in GCS allows modern data warehousing and query engines (like Google BigQuery, Dataproc/Spark) to query the data directly *in situ* without needing to load it into a database first.

### Data Flow Architecture

The data flows from high-frequency access to long-term archival:



## 2. Implementation Strategy

This section outlines the libraries, data models, and a core script for building this system.

### 2.1. Hot Tier: Redis Implementation

- **Library:** `redis-py`
- **Data Modeling:**
  - **Order State:** Use a Redis `HASH`. The key would be `order:{order_id}`.
  - **Live Signals:** Use Redis `STREAMS` for a persistent, fan-out message queue, or `PUBSUB` for a fire-and-forget broadcast. Streams are generally more robust.

```
# Python example for managing Order State in Redis
import redis

# Connect to Redis
r = redis.Redis(host='localhost', port=6379, db=0, decode_responses=True)

def update_order_state(order_id: str, status: str, filled_qty: int):
    """Updates an order's state in Redis using a HASH."""
    order_key = f"order:{order_id}"
    r.hset(order_key, mapping={
        "status": status,
        "filled_qty": filled_qty,
        "last_updated": datetime.utcnow().isoformat()
    })
    # Set an expiration to auto-clean stale orders if necessary
    r.expire(order_key, 3600 * 24)

def get_order_state(order_id: str):
```

```
"""Retrieves an order's state."""  
return r.hgetall(f"order:{order_id}")
```

## 2.2. Warm Tier: PostgreSQL Implementation

- **Library:** `SQLAlchemy` (for ORM or Core) and `psycopg2-binary`.
- **Schema Design:** The tables must be designed for efficient querying and archival. Using a timestamp column is critical for the migration logic.

```
-- SQL Schema for the Trade Ledger  
CREATE TABLE trade_ledger (  
    trade_id BIGSERIAL PRIMARY KEY,  
    order_id VARCHAR(255) NOT NULL,  
    symbol VARCHAR(20) NOT NULL,  
    side VARCHAR(4) NOT NULL, -- 'BUY' or 'SELL'  
    price NUMERIC(18, 8) NOT NULL,  
    quantity NUMERIC(18, 8) NOT NULL,  
    trade_timestamp TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
    -- Other relevant fields: fees, exchange_trade_id, etc.  
);  
  
-- Create an index on the timestamp for fast archival queries  
CREATE INDEX idx_trade_ledger_timestamp ON trade_ledger (trade_timestamp);
```

## 2.3. Cold Tier & Migration Script

This script is the core of the data lifecycle management. It runs nightly to archive data older than 30 days from PostgreSQL to GCS.

- **Libraries:** `pandas`, `sqlalchemy`, `pyarrow`, `google-cloud-storage`.
- **Format:** Parquet. It offers superior compression and query performance over CSV.
- **Logic:**
  1. Establish connections to PostgreSQL and GCS.
  2. Calculate the cutoff date (30 days prior to the current date).

3. Query PostgreSQL for records older than the cutoff, fetching them in manageable chunks to avoid high memory usage.
4. Convert each chunk to a Pandas DataFrame.
5. Save the DataFrame as a Parquet file locally.
6. Upload the Parquet file to a GCS bucket with a structured name (e.g., `gs://<bucket>/trades/year=YYYY/month=MM/day=DD/data.parquet` ).
7. **Crucially**, upon successful upload, delete the corresponding records from the PostgreSQL table within a transaction.

## Python Nightly Migration Script

```
import os
import pandas as pd
from sqlalchemy import create_engine, text
from google.cloud import storage
from datetime import datetime, timedelta

# --- Configuration ---
# Best practice: Use environment variables or a config management system
DB_CONNECTION_STRING = os.environ.get("DB_CONNECTION_STRING") # e.g.,
"postgresql://user:pass@host:port/dbname"
GCS_BUCKET_NAME = os.environ.get("GCS_BUCKET_NAME")
GCS_CREDENTIALS_PATH = os.environ.get("GOOGLE_APPLICATION_CREDENTIALS") # Path
to service account JSON

# --- Constants ---
DAYS_TO_RETAIN = 30
CHUNK_SIZE = 50000 # Process 50,000 rows at a time to manage memory

def migrate_postgres_to_gcs():
    """
    Archives data older than DAYS_TO_RETAIN from Postgres to GCS in Parquet
    format.
    """
    print("Starting nightly data migration...")

    # 1. Establish connections
    try:
```

```

        engine = create_engine(DB_CONNECTION_STRING)
        storage_client =
storage.Client.from_service_account_json(GCS_CREDENTIALS_PATH)
        bucket = storage_client.bucket(GCS_BUCKET_NAME)
    except Exception as e:
        print(f"Error establishing connections: {e}")
        return

    # 2. Calculate cutoff date
    cutoff_date = datetime.utcnow() - timedelta(days=DAYS_TO_RETAIN)
    cutoff_str = cutoff_date.strftime('%Y-%m-%d %H:%M:%S')
    print(f"Archiving data older than: {cutoff_str}")

    table_to_archive = "trade_ledger"
    timestamp_column = "trade_timestamp"

    # 3. Query and process in chunks
    query = f"""
        SELECT trade_id FROM {table_to_archive}
        WHERE {timestamp_column} < :cutoff_date
        ORDER BY trade_id
    """

    with engine.connect() as conn:
        # Use a server-side cursor for large datasets if needed
        result_proxy = conn.execute(text(query), {"cutoff_date": cutoff_str})

        while True:
            # Fetch a chunk of IDs to process
            id_chunk = [row[0] for row in result_proxy.fetchmany(CHUNK_SIZE)]
            if not id_chunk:
                break # No more data to process

            print(f"Processing {len(id_chunk)} records...")

            # Fetch full data for the chunk
            data_query = f"SELECT * FROM {table_to_archive} WHERE trade_id
IN :ids"
            df = pd.read_sql(text(data_query), conn, params={"ids":
tuple(id_chunk)})

```

```

        if df.empty:
            continue

        # 4. Convert to Parquet and upload to GCS
        archive_date = df[timestamp_column].min().date()
        gcs_path = f"archives/{table_to_archive}/year={archive_date.year}/
month={archive_date.month:02d}/{table_to_archive}
_{archive_date.strftime('%Y%m%d')}.parquet"

        try:
            # Save to a local temp file first
            local_path = "/tmp/archive.parquet"
            df.to_parquet(local_path, index=False, engine='pyarrow')

            # Upload
            blob = bucket.blob(gcs_path)
            blob.upload_from_filename(local_path)
            print(f"Successfully uploaded {len(df)} records to {gcs_path}")
            os.remove(local_path)

            # 5. Delete archived data from PostgreSQL in a transaction
            with conn.begin(): # Starts a transaction
                delete_query = f"DELETE FROM {table_to_archive} WHERE
trade_id IN :ids"
                conn.execute(text(delete_query), {"ids": tuple(id_chunk)})
                print(f"Successfully deleted {len(id_chunk)} records from
PostgreSQL.")

        except Exception as e:
            print(f"An error occurred during chunk processing: {e}")
            # The transaction will be rolled back automatically on error
            # Consider adding retry logic or dead-letter queueing
            return # Stop the process on failure to avoid data loss

    print("Nightly data migration completed.")

if __name__ == "__main__":
    # This script can be scheduled with Cron, Airflow, or Google Cloud Scheduler
    migrate_postgres_to_gcs()

```



---

## 3. Critical Analysis

### 3.1. Potential Failure Modes & Edge Cases

- **Migration Script Failure:** If the script fails after uploading to GCS but before deleting from Postgres, data will be duplicated. If it fails after deleting but before a successful upload, data will be lost.
  - **Mitigation:** The provided script uses a transaction for the delete operation. However, the entire process is not atomic. A more robust solution would involve a two-phase commit system or a "soft delete" (marking rows for deletion and purging them later) after verifying the GCS upload. Implementing idempotency (e.g., checking if the target GCS file already exists) is also crucial.
- **Data Consistency between Tiers:** The architecture doesn't explicitly state how data moves from Redis to Postgres (e.g., when an order is filled). This is a critical pathway.
  - **Mitigation:** A reliable mechanism is needed. Options include:
    1. **Application-Level Dual Writes:** The application writes to both Redis and Postgres. This risks inconsistency if one write fails.
    2. **Change Data Capture (CDC):** Use a tool like Debezium to stream changes from Redis (if using RedisGears/modules) or, more commonly, from the primary application's write-ahead-log into Postgres.
    3. **Asynchronous Task Queue:** The application writes to Redis and pushes a message to a queue (e.g., RabbitMQ, Celery). A separate worker consumes from the queue and writes to Postgres.
- **"Cache Stampede" on Redis Failure:** If the Redis instance fails, all services will fall back to querying the "warm" Postgres database, potentially overwhelming it.
  - **Mitigation:** Implement a circuit breaker pattern in the application logic. Additionally, have a clear strategy for repopulating the Redis cache upon recovery to avoid all services hitting it at once.

- **Backpressure:** A sudden spike in trading volume could overwhelm the Redis -> Postgres write pathway, causing delays and data lag in the warm tier.
  - **Mitigation:** The chosen pathway (e.g., a task queue) must be able to handle backpressure, either by buffering, scaling workers, or gracefully shedding load.

## 3.2. Optimizations & Enhancements

- **PostgreSQL Partitioning:** For tables like `trade_ledger`, performance will degrade as they grow, even with only 30 days of data. The nightly `DELETE` operation can become very slow and cause table bloat.
  - **Optimization:** Partition the table by date range (e.g., daily or weekly partitions). Instead of a `DELETE` operation, the script can simply `DETACH` and `DROP` an entire old partition, which is an instantaneous, non-blocking metadata operation. This is a massive performance gain.
- **GCS Lifecycle Management:** To further reduce costs, configure a GCS Lifecycle Policy to automatically transition archived data to cheaper storage classes (e.g., Nearline, Coldline, or Archive) after a certain period (e.g., 1 year).
- **Querying Cold Data:** To make the GCS archives useful, use a query engine that can read them directly.
  - **Optimization:** Register the GCS bucket as an external table in **Google BigQuery**. This allows analysts to run complex SQL queries over terabytes of historical Parquet data without importing it, paying only for the data scanned.
- **Compression and File Sizing:** For the Parquet files, use an efficient compression codec like Snappy or ZSTD. Aim for optimal file sizes in GCS (typically 256MB - 1GB) to maximize read parallelism in query engines like BigQuery or Spark. The script can be adapted to aggregate data into fewer, larger files.